

SDD — Módulo 2: Estructuras Jerárquicas

1. Información general del proyecto

Nombre del proyecto: Visualizador Web de Estructuras de Datos

Módulo: Estructuras Jerárquicas

Versión: 0.1.0

Estado: Especificación inicial para implementación del segundo módulo

Tecnologías: Python, Flask, HTML5, CSS3, JavaScript, Jinja2

Tipo de proyecto: Aplicación web didáctica

Público objetivo: Estudiantes, docentes y monitores de estructuras de datos

Descripción general

El módulo de estructuras jerárquicas permitirá a los estudiantes interactuar con árboles y montículos, ejecutar operaciones sobre ellos y visualizar su comportamiento interno mediante representaciones gráficas jerárquicas.

Este módulo debe integrarse con la arquitectura general del proyecto y conservar las mismas especificaciones de calidad, separación de capas, documentación, ayudas didácticas, trazabilidad, pruebas y criterios de aceptación definidos para el módulo de estructuras secuenciales.

Objetivo del módulo

Diseñar e implementar un módulo web didáctico que permita seleccionar una estructura jerárquica, ejecutar operaciones válidas sobre ella, visualizar su estado interno y comprender sus reglas mediante ayuda contextual.

Alcance inicial

La primera versión del módulo jerárquico debe incluir los siguientes TAD:

- Árbol Binario de Búsqueda, ABB.
- Árbol AVL.
- Montículo Binario.
- Árbol Rojo-Negro.
- Excepciones comunes reutilizadas desde el dominio.

Fuera de alcance para esta fase

- Grafos.
- Union-Find.
- Árboles B/B+.
- Trie.
- Persistencia en base de datos.
- Autenticación de usuarios.
- Evaluación automática de estudiantes.
- Implementaciones en C, aunque la arquitectura debe quedar preparada para ellas.

2. Visión del sistema

Problema que resuelve

Las estructuras jerárquicas suelen ser difíciles de comprender porque involucran relaciones padre-hijo, profundidad, altura, rotaciones, balanceo, colores y recorridos. El módulo busca convertir estos conceptos en experiencias visuales e interactivas.

Usuarios objetivo

- Estudiantes de estructuras de datos.
- Docentes de programación.
- Monitores académicos.
- Desarrolladores educativos.

Beneficios esperados

- Comprensión visual de árboles y montículos.
- Observación paso a paso de inserciones, eliminaciones y recorridos.
- Explicación didáctica de balanceo en AVL y Rojo-Negro.
- Visualización de propiedades como altura, raíz, hojas, mínimo, máximo y tamaño.
- Base extensible para futuras estructuras jerárquicas y versiones en C.

Contexto de uso

El módulo se usará desde un navegador web dentro de la misma aplicación Flask del proyecto principal.

3. Requisitos funcionales

ID	Requisito	Descripción	Prioridad	Actor
RF-J-001	Acceso al módulo	Permitir ingresar al módulo de estructuras jerárquicas desde la página principal	Alta	Estudiante
RF-J-002	Listado de estructuras	Mostrar tarjetas para ABB, AVL, Montículo Binario y Rojo-Negro	Alta	Estudiante
RF-J-003	Selección de estructura	Permitir seleccionar una estructura jerárquica específica	Alta	Estudiante
RF-J-004	Visualización jerárquica	Mostrar nodos, relaciones padre-hijo, raíz, hojas y niveles	Alta	Estudiante
RF-J-005	Ejecución de operaciones	Ejecutar solo operaciones válidas según el TAD seleccionado	Alta	Estudiante
RF-J-006	Inserción	Permitir insertar valores y mostrar el efecto en la estructura	Alta	Estudiante
RF-J-007	Eliminación	Permitir eliminar valores si el TAD lo soporta	Alta	Estudiante

ID	Requisito	Descripción	Prioridad	Actor
RF-J-008	Búsqueda	Permitir buscar un valor y resaltar el recorrido o resultado	Alta	Estudiante
RF-J-009	Recorridos	Mostrar recorridos disponibles, como inorden, preorden y postorden cuando existan	Alta	Estudiante
RF-J-010	Propiedades	Mostrar altura, tamaño, mínimo, máximo y validación cuando existan	Media	Estudiante
RF-J-011	Balanceo AVL	Mostrar explicación didáctica de balanceo en AVL después de insertar o eliminar	Alta	Estudiante
RF-J-012	Colores Rojo-Negro	Representar visualmente nodos rojos y negros	Alta	Estudiante
RF-J-013	Montículo por arreglo	Mostrar el montículo como árbol y como arreglo interno	Alta	Estudiante
RF-J-014	Ayuda del módulo	Proveer ayuda general sobre estructuras jerárquicas	Alta	Estudiante
RF-J-015	Ayuda por estructura	Proveer ayuda individual para ABB, AVL, Montículo Binario y Rojo-Negro	Alta	Estudiante
RF-J-016	Manejo de errores	Traducir excepciones técnicas a mensajes didácticos	Alta	Estudiante
RF-J-017	Estado de sesión	Mantener el estado temporal de la estructura seleccionada	Media	Estudiante
RF-J-018	Separación lógica/visual	Mantener separados TAD, adaptadores, servicios, rutas, plantillas y JS	Alta	Desarrollador
RF-J-019	Preparación para C	Mantener contratos de adaptadores compatibles con futuras implementaciones en C	Alta	Desarrollador
RF-J-020	Plantillas reutilizables	Usar el mismo header, footer, navbar y layout base del proyecto	Alta	Desarrollador

4. Requisitos no funcionales

ID	Tipo	Descripción
RNF-J-001	Documentación	Todo el código debe estar documentado según estándares del lenguaje
RNF-J-002	Python	El código Python debe cumplir PEP8, usar docstrings y type hints cuando sea razonable

ID	Tipo	Descripción
RNF-J-003	JavaScript	Separar eventos, renderizado, animaciones y comunicación con backend
RNF-J-004	HTML	Usar HTML semántico y plantillas Jinja2 reutilizables
RNF-J-005	CSS	Separar estilos base, layout, componentes y visualización jerárquica
RNF-J-006	Modularidad	Cada estructura debe estar encapsulada mediante adaptadores
RNF-J-007	Mantenibilidad	Debe ser sencillo agregar nuevos árboles en el futuro
RNF-J-008	Seguridad	Validar entradas y evitar ejecución de código arbitrario
RNF-J-009	Usabilidad	La interfaz debe ser clara para estudiantes principiantes
RNF-J-010	Rendimiento	El sistema debe funcionar con árboles pequeños y medianos de uso didáctico
RNF-J-011	Extensibilidad	La integración futura con C no debe afectar la interfaz web
RNF-J-012	Trazabilidad	Cada requisito debe mapearse con casos de uso, módulos y pruebas
RNF-J-013	Accesibilidad	Usar colores, etiquetas y textos alternativos para reforzar la comprensión
RNF-J-014	Consistencia visual	Mantener el mismo sistema visual del módulo secuencial

5. Casos de uso

CU-J-001 — Acceder al módulo jerárquico

Actor: Estudiante

Descripción: El usuario ingresa al módulo de estructuras jerárquicas.

Precondiciones: La aplicación principal está disponible.

Flujo principal:

1. El usuario abre la página principal.
2. Selecciona "Estructuras jerárquicas".
3. El sistema muestra las estructuras disponibles.

Postcondición: El usuario puede seleccionar una estructura jerárquica.

CU-J-002 — Seleccionar estructura jerárquica

Actor: Estudiante

Descripción: El usuario selecciona ABB, AVL, Montículo Binario o Rojo-Negro.

Precondiciones: El usuario está en el módulo jerárquico.

Flujo principal:

1. El usuario selecciona una estructura.
2. El sistema crea o recupera su estado.
3. El sistema carga el panel de operaciones.
4. El sistema muestra una visualización inicial vacía.

Postcondición: La estructura queda activa.

CU-J-003 — Insertar valor

Actor: Estudiante

Descripción: El usuario inserta un valor en la estructura.

Precondiciones: Existe una estructura activa.

Flujo principal:

1. El usuario ingresa un valor.
2. El sistema valida la entrada.
3. El adaptador invoca el método real del TAD.
4. El sistema actualiza la visualización.
5. El sistema muestra explicación didáctica.

Flujos alternativos:

- Si el valor ya existe en estructuras sin duplicados, se muestra un mensaje didáctico.
- Si la entrada no es válida, se solicita corrección.

Postcondición: El valor queda insertado si la operación fue válida.

CU-J-004 — Eliminar valor

Actor: Estudiante

Descripción: El usuario elimina un valor de la estructura.

Precondiciones: La estructura contiene elementos.

Flujo principal:

1. El usuario ingresa el valor a eliminar.
2. El sistema valida la entrada.
3. El adaptador ejecuta la eliminación.
4. El sistema actualiza la visualización.
5. El sistema muestra explicación sobre el cambio producido.

Flujos alternativos:

- Si el valor no existe, se muestra mensaje didáctico.

- Si el TAD no soporta eliminación, la operación no se muestra.

Postcondición: El valor se elimina si existe y si la operación está soportada.

CU-J-005 — Buscar valor

Actor: Estudiante

Descripción: El usuario busca un valor en la estructura.

Precondiciones: Existe una estructura activa.

Flujo principal:

1. El usuario ingresa un valor.
2. El sistema ejecuta la búsqueda.
3. El sistema indica si el valor existe.
4. Si es posible, el sistema resalta el nodo encontrado.

Postcondición: El usuario recibe el resultado de búsqueda.

CU-J-006 — Consultar recorridos

Actor: Estudiante

Descripción: El usuario solicita un recorrido de árbol.

Precondiciones: La estructura soporta recorridos.

Flujo principal:

1. El usuario selecciona un tipo de recorrido.
2. El sistema invoca el método correspondiente.
3. El sistema muestra la lista resultante.
4. El sistema explica el orden visitado.

Postcondición: El usuario visualiza el resultado del recorrido.

CU-J-007 — Consultar propiedades

Actor: Estudiante

Descripción: El usuario consulta propiedades de la estructura.

Precondiciones: Existe una estructura activa.

Flujo principal:

1. El usuario selecciona una propiedad.
2. El sistema calcula o consulta el valor.
3. El sistema muestra el resultado y su significado.

Postcondición: El usuario comprende propiedades como altura, tamaño, mínimo, máximo o validez.

CU-J-008 — Consultar ayuda

Actor: Estudiante

Descripción: El usuario consulta ayuda del módulo o estructura.

Precondiciones: El usuario está en la aplicación.

Flujo principal:

1. El usuario selecciona ayuda.
2. El sistema muestra definición conceptual.
3. El sistema muestra operaciones principales.
4. El sistema muestra complejidad, ejemplos y errores comunes.

Postcondición: El usuario obtiene apoyo didáctico.

6. Modelo de datos

6.1 Diccionario de datos

Entidad: HierarchicalStructureSession

Atributo	Tipo	Descripción
session_id	string	Identificador de sesión
module	string	Valor fijo: hierarchical
structure_type	string	Tipo de estructura jerárquica
state	dict/list	Estado serializable
last_operation	string	Última operación ejecutada
last_result	dict/string	Resultado de la última operación
errors	list	Errores recientes

Entidad: HierarchicalStructureDefinition

Atributo	Tipo	Descripción
id	string	Identificador interno
name	string	Nombre visible
description	string	Descripción didáctica
operations	list	Operaciones disponibles
help_content	dict	Contenido de ayuda
adapter_class	string	Adaptador asociado
visualizer_type	string	Tipo de visualizador requerido

Entidad: TreeNodeVisualState

Atributo	Tipo	Descripción
id	string	Identificador único del nodo visual
value	string/number	Valor almacenado en el nodo
left	string/null	ID del hijo izquierdo, si existe
right	string/null	ID del hijo derecho, si existe
parent	string/null	ID del padre, si aplica
level	int	Nivel dentro del árbol
height	int/null	Altura del nodo, si aplica
balance_factor	int/null	Factor de balance, si aplica
color	string/null	Color para árbol rojo-negro
highlighted	bool	Indica si el nodo debe resaltarse

Entidad: HeapVisualState

Atributo	Tipo	Descripción
items	list	Representación interna del arreglo
root	object/null	Raíz del montículo
size	int	Cantidad de elementos
heap_type	string	min_heap o max_heap
tree_nodes	list	Nodos derivados del arreglo

7. Arquitectura del sistema

7.1 Estilo arquitectónico

El módulo jerárquico debe seguir la misma arquitectura por capas del proyecto:

```
Usuario
  ↓
Frontend HTML/CSS/JavaScript
  ↓
Rutas Flask / Controladores
  ↓
Servicios de aplicación
  ↓
Adaptadores de estructuras jerárquicas
```


↓
TAD Python actuales
↓
Futuras implementaciones en C

7.2 Capas principales

Capa de presentación

Responsable de:

- Renderizar plantillas del módulo jerárquico.
- Dibujar árboles y montículos.
- Mostrar animaciones de inserción, eliminación, búsqueda y recorridos.
- Mostrar ayuda y mensajes didácticos.

Capa de rutas Flask

Responsable de:

- Recibir solicitudes HTTP.
- Validar datos básicos.
- Invocar servicios.
- Retornar HTML o JSON.

No debe contener lógica interna de árboles o montículos.

Capa de servicios

Responsable de:

- Coordinar operaciones.
- Manejar estado de sesión.
- Capturar excepciones.
- Traducir respuestas a mensajes didácticos.
- Entregar estados serializables al frontend.

Capa de adaptadores

Responsable de normalizar el acceso a los TAD jerárquicos.

Adaptadores previstos:

- BinarySearchTreeAdapter.
- AVLTreeAdapter.
- BinaryHeapAdapter.
- RedBlackTreeAdapter.

Capa de dominio

Contiene los TAD originales:

- `abb.py`.
- `avl.py`.
- `monticulo_binario.py`.
- `rojo_negro.py`.
- `exceptions.py`.

Esta capa debe permanecer independiente de Flask, plantillas y JavaScript.

8. Estructura de carpetas propuesta

```
visualizador_estructuras/  
├── app/  
│   ├── routes/  
│   │   ├── hierarchical_routes.py  
│   │   └── help_routes.py  
│   ├── services/  
│   │   ├── hierarchical_service.py  
│   │   ├── session_service.py  
│   │   └── help_service.py  
│   ├── adapters/  
│   │   ├── base_adapter.py  
│   │   ├── binary_search_tree_adapter.py  
│   │   ├── avl_tree_adapter.py  
│   │   ├── binary_heap_adapter.py  
│   │   └── red_black_tree_adapter.py  
│   ├── domain/  
│   │   ├── hierarchical/  
│   │   │   ├── __init__.py  
│   │   │   ├── abb.py  
│   │   │   ├── avl.py  
│   │   │   ├── monticulo_binario.py  
│   │   │   └── rojo_negro.py  
│   │   └── common/  
│   │       └── exceptions.py  
│   └── templates/  
│       ├── base.html  
│       ├── partials/  
│       │   └── header.html
```

```
|
|
|   ├── footer.html
|   ├── navbar.html
|   ├── messages.html
|   └── help_panel.html
|
|   ├── pages/
|   │   ├── hierarchical.html
|   │   └── hierarchical_structure_view.html
|   └── help/
|       ├── hierarchical_help.html
|       ├── abb_help.html
|       ├── avl_help.html
|       ├── heap_help.html
|       └── red_black_help.html
|
|   └── static/
|       ├── css/
|       │   ├── hierarchical.css
|       │   └── tree_visualizer.css
|       ├── js/
|       │   ├── hierarchical_api_client.js
|       │   ├── tree_visualizer.js
|       │   ├── heap_visualizer.js
|       │   ├── tree_animations.js
|       │   └── hierarchical_helpers.js
|       └── img/
|
|   ├── tests/
|   │   ├── test_binary_search_tree_adapter.py
|   │   ├── test_avl_tree_adapter.py
|   │   ├── test_binary_heap_adapter.py
|   │   ├── test_red_black_tree_adapter.py
|   │   ├── test_hierarchical_routes.py
|   │   └── test_hierarchical_visual_state.py
|   └── docs/
|       ├── hierarchical_sdd.md
|       ├── hierarchical_user_guide.md
|       └── future_c_integration.md
```

9. Contratos de interfaz UI/UX

Pantalla del módulo jerárquico

Debe incluir:

- Título: "Estructuras jerárquicas".
- Descripción general.
- Tarjetas para ABB, AVL, Montículo Binario y Rojo-Negro.
- Estado de cada estructura.
- Botón "Explorar".
- Botón "Ayuda del módulo".

Pantalla de visualización jerárquica

Header / Navbar	
Nombre de estructura	
Descripción breve	
Panel de operaciones	Visualización jerárquica
Propiedades: tamaño, altura, raíz, etc.	
Explicación paso a paso	
Resultado / errores	
Footer	

Contrato visual para árboles

```
{
  "structure": "avl",
  "nodes": [
    {
      "id": "node-10",
      "value": 10,
      "left": "node-5",
      "right": "node-15",
      "parent": null,
      "level": 0,
      "height": 2,
      "balance_factor": 0,
      "color": null,
      "highlighted": true
    }
  ],
}
```

```

    "edges": [
      {"from": "node-10", "to": "node-5", "label": "izq"},
      {"from": "node-10", "to": "node-15", "label": "der"}
    ],
    "metadata": {
      "root": 10,
      "size": 3,
      "height": 2,
      "is_empty": false,
      "is_valid": true
    },
    "last_operation": {
      "name": "insertar",
      "value": 10,
      "status": "success",
      "message": "El valor 10 fue insertado y el árbol conserva sus
propiedades."
    }
  }
}

```

Contrato visual para montículo

```

{
  "structure": "binary_heap",
  "items": [1, 4, 3, 8, 6],
  "nodes": [
    {"id": "heap-0", "value": 1, "index": 0, "left_index": 1, "right_index":
2}
  ],
  "metadata": {
    "root": 1,
    "size": 5,
    "heap_type": "min_heap",
    "is_empty": false
  },
  "last_operation": {
    "name": "insertar",
    "value": 1,
    "status": "success",
    "message": "El valor fue insertado y subió hasta conservar la propiedad
de montículo."
  }
}

```

10. Reglas de negocio

ID	Regla	Descripción
RB-J-001	Operaciones válidas	Solo se deben mostrar operaciones soportadas por el TAD seleccionado
RB-J-002	Sin duplicados	ABB, AVL y Rojo-Negro no deben aceptar valores duplicados
RB-J-003	Estructura vacía	No se deben ejecutar consultas como mínimo, máximo o extracción sobre estructuras vacías
RB-J-004	Mensajes didácticos	Todo error técnico debe traducirse a lenguaje comprensible
RB-J-005	Dominio independiente	Los TAD no deben importar Flask ni depender de la interfaz
RB-J-006	Estado serializable	El backend debe entregar estados JSON serializables
RB-J-007	Contrato estable	Los nombres internos de operaciones deben mantenerse estables
RB-J-008	Integración futura con C	El cambio a C debe ocurrir detrás de adaptadores
RB-J-009	No inventar operaciones	Si una operación no existe en el TAD, debe marcarse como pendiente
RB-J-010	Validación de entradas	Todo valor ingresado debe validarse antes de ejecutar operaciones
RB-J-011	Propiedades visuales	AVL debe mostrar altura y balance cuando sea posible
RB-J-012	Colores visuales	Rojo-Negro debe representar nodos rojos y negros
RB-J-013	Doble representación	Montículo debe poder mostrarse como árbol y como arreglo

11. Operaciones iniciales por TAD

Árbol Binario de Búsqueda, ABB

Operaciones esperadas:

- `insertar(dato)`.
- `eliminar(dato)`.
- `buscar(dato)`.
- `minimo()`.

- `maximo()` .
- `altura()` .
- `contar_hojas()` .
- `inorden()` .
- `preorden()` .
- `postorden()` .
- `validar()` .
- `limpiar()` .
- `vacio()` .
- `tamano()` .

Árbol AVL

Operaciones esperadas:

- `insertar(dato)` .
- `eliminar(dato)` .
- `buscar(dato)` .
- `minimo()` .
- `maximo()` .
- `altura()` .
- `inorden()` .
- `validar()` .
- `limpiar()` .
- `vacio()` .
- `tamano()` .

Observación didáctica:

El adaptador debe explicar que el AVL realiza balanceo automático mediante rotaciones. Si el TAD no expone directamente el tipo de rotación realizada, el sistema debe limitarse a indicar que el árbol fue rebalanceado o que conserva la propiedad AVL, sin inventar una rotación específica.

Montículo Binario

Operaciones esperadas:

- `insertar(dato)` .
- `extraer_raiz()` .
- `raiz()` .
- `vacio()` .
- `tamano()` .
- `limpiar()` .
- `a_lista()` .

Observación didáctica:

El montículo debe mostrarse tanto como arreglo interno como árbol conceptual. El adaptador no debe asumir orden total de recorrido de árbol; debe respetar la representación interna del arreglo.

Árbol Rojo-Negro

Operaciones esperadas:

- `insertar(dato)`.
- `eliminar(dato)`.
- `buscar(dato)`.
- `inorden()`.
- `altura()`.
- `validar()`.
- `limpiar()`.
- `vacio()`.
- `tamano()`.

Observación didáctica:

El adaptador debe representar colores rojo y negro. Si el TAD no expone una API pública para obtener nodos y colores, el agente debe crear un mecanismo de visualización respetando la encapsulación tanto como sea posible o marcar la limitación como pendiente técnica.

12. Especificación de algoritmos

Algoritmo general para ejecutar una operación jerárquica

Entrada:

- Tipo de estructura.
- Operación solicitada.
- Parámetros del usuario.
- Estado actual de sesión.

Proceso:

1. Validar que la estructura exista.
2. Validar que la operación pertenezca al conjunto soportado.
3. Validar parámetros de entrada.
4. Recuperar instancia desde sesión.
5. Invocar el adaptador correspondiente.
6. Ejecutar el método real del TAD.
7. Capturar excepciones del dominio.
8. Traducir resultado a mensaje didáctico.
9. Generar estado visual serializable.
10. Retornar respuesta HTML o JSON a la UI.

Salida:

- Estado actualizado.
- Resultado de operación.
- Mensaje didáctico.
- Datos para visualización jerárquica.

Algoritmo para renderizar árbol

Entrada: Estado serializado del árbol.

Proceso:

1. Limpiar contenedor visual.
2. Calcular niveles de los nodos.
3. Posicionar raíz en el nivel superior.
4. Posicionar hijos izquierdos y derechos.
5. Dibujar aristas padre-hijo.
6. Aplicar colores, etiquetas y resaltados.
7. Mostrar metadatos como altura, tamaño y raíz.

Salida: Árbol dibujado en pantalla.

Algoritmo para renderizar montículo

Entrada: Arreglo interno del montículo.

Proceso:

1. Recibir lista de elementos.
2. Para cada índice i , calcular:
3. hijo izquierdo: $2i + 1$.
4. hijo derecho: $2i + 2$.
5. Crear nodos visuales.
6. Dibujar relaciones padre-hijo.
7. Mostrar también el arreglo interno.
8. Resaltar raíz y último elemento afectado.

Salida: Montículo representado como árbol y arreglo.

13. API / servicios

Rutas web sugeridas

Endpoint	Método	Descripción
<code>/jerarquicas</code>	GET	Página del módulo jerárquico
<code>/jerarquicas/<structure_id></code>	GET	Página de visualización de una estructura jerárquica
<code>/api/jerarquicas/<structure_id>/operation</code>	POST	Ejecutar operación

Endpoint	Método	Descripción
<code>/api/jerarquicas/<structure_id>/state</code>	GET	Obtener estado actual serializado
<code>/api/jerarquicas/<structure_id>/reset</code>	POST	Reiniciar estructura
<code>/ayuda/jerarquicas</code>	GET	Ayuda general del módulo
<code>/ayuda/jerarquicas/<structure_id></code>	GET	Ayuda específica por estructura

Servicio: HierarchicalService

Responsabilidades:

- Listar estructuras jerárquicas disponibles.
- Crear estructuras.
- Ejecutar operaciones.
- Consultar estado.
- Reiniciar estructura.

Métodos conceptuales:

```
get_available_structures()
create_structure(structure_type)
execute_operation(structure_type, operation, payload)
get_structure_state(structure_type)
reset_structure(structure_type)
```

Servicio: SessionService

Responsabilidades:

- Guardar estado temporal.
- Recuperar estructura activa.
- Limpiar sesión.
- Serializar y deserializar datos.

Servicio: HelpService

Responsabilidades:

- Obtener ayuda por módulo.
- Obtener ayuda por estructura.
- Obtener descripción de operaciones.
- Proveer complejidades y errores comunes.

Contrato de adaptador base

Cada adaptador debe cumplir una interfaz común:

```

create()
execute(operation_name, payload)
to_visual_state()
reset()
get_supported_operations()

```

14. Trazabilidad

RF	Caso de uso	Módulo	Prueba
RF-J-001	CU-J-001	Hierarchical Routes	test_hierarchical_module_loads
RF-J-002	CU-J-001	HierarchicalService	test_available_hierarchical_structures
RF-J-003	CU-J-002	Hierarchical Routes	test_structure_selection
RF-J-004	CU-J-002 a CU-J-007	Tree Visualizer JS	test_tree_visual_state_response
RF-J-005	CU-J-003 a CU-J-007	Adapters	test_adapter_supported_operations
RF-J-006	CU-J-003	Adapters	test_insert_operation
RF-J-007	CU-J-004	Adapters	test_delete_operation
RF-J-008	CU-J-005	Adapters	test_search_operation
RF-J-009	CU-J-006	Adapters	test_traversal_operations
RF-J-010	CU-J-007	Adapters	test_tree_properties
RF-J-011	CU-J-003, CU-J-004	AVLTreeAdapter	test_avl_validation_after_changes
RF-J-012	CU-J-003, CU-J-004	RedBlackTreeAdapter	test_red_black_visual_colors
RF-J-013	CU-J-003, CU-J-007	BinaryHeapAdapter	test_heap_array_and_tree_state

RF	Caso de uso	Módulo	Prueba
RF-J-014	CU-J-008	HelpService	test_hierarchical_help_available
RF-J-015	CU-J-008	HelpService	test_structure_help_available
RF-J-016	CU-J-003 a CU-J-007	Services / Exceptions	test_hierarchical_error_messages
RF-J-017	CU-J-003 a CU-J-007	SessionService	test_hierarchical_session_persistence
RF-J-018	Todos	Domain / Services / Adapters	test_no_flask_dependency_in_hierarchical_domain
RF-J-019	Futuro	Adapter Layer	test_hierarchical_adapter_contract_stability
RF-J-020	Todos	Templates	test_base_template_reuse

15. Plan de pruebas

Pruebas unitarias

Deben cubrir:

- Adaptadores jerárquicos.
- Servicios.
- Conversión a estado visual.
- Manejo de errores.
- Validaciones de entrada.
- Operaciones por TAD.

Pruebas de integración

Deben validar:

- Rutas Flask del módulo jerárquico.
- Flujo completo de selección de estructura.
- Inserción, eliminación y búsqueda.
- Respuesta JSON/HTML.
- Persistencia temporal en sesión.

Pruebas de interfaz

Deben validar:

- Carga de plantillas.
- Botones visibles.
- Visualización de nodos y aristas.
- Visualización de colores en Rojo-Negro.
- Visualización de arreglo y árbol en Montículo.
- Mensajes de ayuda.

Pruebas didácticas

Deben validar que:

- Cada operación tenga explicación.
- Los errores sean comprensibles.
- Cada estructura tenga ayuda.
- Las propiedades del árbol sean visibles.
- Las operaciones de balanceo no inventen información no disponible.

16. Criterios de aceptación

La primera versión del módulo jerárquico será aceptada cuando:

1. La aplicación Flask mantenga la navegación general.
2. Exista una página del módulo de estructuras jerárquicas.
3. Se puedan seleccionar ABB, AVL, Montículo Binario y Rojo-Negro.
4. Cada estructura tenga una interfaz de operaciones.
5. Las operaciones disponibles se obtengan desde adaptadores.
6. El estado visual se actualice después de cada operación.
7. Los árboles se visualicen con nodos y relaciones padre-hijo.
8. El Montículo Binario se visualice como arreglo y árbol.
9. El Árbol Rojo-Negro represente colores cuando sea técnicamente posible.
10. Los errores sean controlados y explicados de forma didáctica.
11. El código de dominio no importe Flask.
12. Las plantillas reutilicen header, footer y navbar del proyecto.
13. Exista ayuda didáctica del módulo y de cada estructura.
14. El código Python cumpla PEP8 y tenga docstrings.
15. El frontend esté separado en archivos CSS y JavaScript.
16. Existan pruebas iniciales del módulo.
17. El proyecto quede preparado para futuras implementaciones de TAD en C.

17. Gestión del proyecto

Metodología

Desarrollo basado en especificaciones, con implementación incremental por estructuras.

Herramientas sugeridas

- Python 3.
- Flask.
- Pytest.
- HTML5.
- CSS3.
- JavaScript.
- Git.
- Agente IA de desarrollo.

Fases

Fase 1 — Análisis de TAD jerárquicos

- Revisar ABB, AVL, Montículo Binario y Rojo-Negro.
- Extraer clases, métodos, alias y excepciones reales.
- Identificar limitaciones de visualización.

Fase 2 — Base del módulo

- Crear rutas jerárquicas.
- Crear plantillas del módulo.
- Integrar navegación desde la página principal.

Fase 3 — Adaptadores

- Crear adaptador base.
- Crear adaptador para ABB.
- Crear adaptador para AVL.
- Crear adaptador para Montículo Binario.
- Crear adaptador para Rojo-Negro.

Fase 4 — Visualización

- Crear visualizador de árboles.
- Crear visualizador de montículos.
- Crear representación de nodos, aristas, niveles y metadatos.
- Crear resaltado de operaciones.

Fase 5 — Ayuda didáctica

- Crear ayuda general del módulo.
- Crear ayuda para cada estructura.
- Incluir complejidad, casos de uso, errores comunes y ejemplos.

Fase 6 — Pruebas y documentación

- Agregar pruebas unitarias.
- Agregar pruebas de rutas.
- Documentar arquitectura.
- Documentar uso del módulo.
- Documentar preparación para C.

18. Integración con agentes de IA

18.1 Roles

Agente	Responsabilidad
Agente arquitecto	Mantener arquitectura, trazabilidad y separación de capas
Agente backend	Implementar Flask, servicios, adaptadores y pruebas backend
Agente frontend	Implementar plantillas, CSS, JS y visualizaciones jerárquicas
Agente documental	Crear ayudas didácticas, README, guías y docstrings
Agente QA	Crear pruebas unitarias, integración y validación de aceptación

18.2 Prompt base para agente desarrollador

Actúa como un agente desarrollador senior especializado en Python, Flask, HTML5, CSS3, JavaScript y arquitectura limpia para aplicaciones educativas.

Contexto del proyecto:

Debes construir el segundo módulo de una aplicación web didáctica llamada "Visualizador Web de Estructuras de Datos". El objetivo del módulo es permitir que estudiantes aprendan estructuras jerárquicas mediante visualizaciones interactivas.

La aplicación general tiene tres módulos:

1. Estructuras secuenciales.
2. Estructuras jerárquicas.
3. Grafos.

En esta fase solo debes implementar el módulo de estructuras jerárquicas.

TAD disponibles inicialmente para este módulo:

- abb.py
- avl.py
- monticulo_binario.py
- rojo_negro.py
- exceptions.py

Objetivo técnico:

Crear un módulo Flask modular que permita seleccionar una estructura jerárquica, ejecutar operaciones sobre ella, visualizar su estado y mostrar ayuda didáctica. La lógica de los TAD debe estar completamente separada de la capa visual.

Restricciones obligatorias:

1. No mezclar lógica de estructuras con rutas Flask ni plantillas HTML.
2. No modificar los TAD originales salvo que sea estrictamente necesario y documentado.

3. No inventar operaciones que no existan en los TAD. Si una operación requerida no existe, marcarla como pendiente.
4. Todo código Python debe cumplir PEP8.
5. Todo módulo, clase y función Python debe tener docstrings.
6. Usar nombres claros y type hints cuando sea razonable.
7. No usar eval.
8. Validar entradas de usuario.
9. Todos los errores deben traducirse a mensajes didácticos.
10. La interfaz debe usar plantillas reutilizables con header, footer, navbar y layout base.
11. El frontend debe estar separado en HTML, CSS y JavaScript.
12. La arquitectura debe quedar preparada para que en una fase futura los TAD puedan implementarse en C.
13. No implementar grafos ni Union-Find en esta fase.
14. No asumir rotaciones específicas si el TAD no expone esa información.
15. No exponer directamente objetos internos no serializables al frontend.

Arquitectura esperada:

Implementa una arquitectura por capas:

- Capa de presentación:
 - templates/
 - static/css/
 - static/js/
- Capa de rutas:
 - app/routes/hierarchical_routes.py
 - app/routes/help_routes.py
- Capa de servicios:
 - app/services/hierarchical_service.py
 - app/services/session_service.py
 - app/services/help_service.py
- Capa de adaptadores:
 - app/adapters/base_adapter.py
 - app/adapters/binary_search_tree_adapter.py
 - app/adapters/avl_tree_adapter.py
 - app/adapters/binary_heap_adapter.py
 - app/adapters/red_black_tree_adapter.py
- Capa de dominio:
 - app/domain/hierarchical/
 - Allí deben ubicarse los TAD suministrados.

Funcionalidades mínimas:

1. Página del módulo jerárquico.
2. Navegación desde la página principal.
3. Tarjetas para ABB, AVL, Montículo Binario y Rojo-Negro.
4. Página de visualización para cada estructura.
5. Panel de operaciones dinámico según la estructura seleccionada.

6. Visualización gráfica jerárquica.
7. Visualización de arreglo y árbol para Montículo Binario.
8. Representación visual de colores para Rojo-Negro cuando sea técnicamente posible.
9. Mensajes didácticos después de cada operación.
10. Manejo de errores para duplicados, elementos inexistentes y estructuras vacías.
11. Sección de ayuda general para estructuras jerárquicas.
12. Ayuda específica para cada estructura.

Contrato de adaptador:

Cada adaptador debe implementar una interfaz común equivalente a:

- create()
- execute(operation_name, payload)
- to_visual_state()
- reset()
- get_supported_operations()

El método to_visual_state() debe devolver datos serializables, por ejemplo:

```
{
  "structure": "avl",
  "nodes": [],
  "edges": [],
  "metadata": {
    "root": null,
    "size": 0,
    "height": 0,
    "is_empty": true,
    "is_valid": true
  },
  "last_operation": {
    "name": "insertar",
    "status": "success",
    "message": "Operación ejecutada correctamente."
  }
}
```

Pruebas requeridas:

Crea pruebas para:

1. Carga del módulo jerárquico.
2. Listado de estructuras disponibles.
3. Creación de adaptadores.
4. Operaciones válidas por estructura.
5. Manejo de duplicados.
6. Manejo de elementos inexistentes.
7. Manejo de estructuras vacías.
8. Conversión a estado visual.
9. Persistencia temporal del estado en sesión.
10. Disponibilidad de ayuda didáctica.

11. Validación de ABB, AVL y Rojo-Negro después de operaciones.
12. Representación de Montículo Binario como arreglo y árbol.

Criterios de aceptación:

La tarea se considera completa si:

1. La aplicación Flask ejecuta sin errores.
2. El módulo jerárquico está disponible desde la navegación.
3. ABB, AVL, Montículo Binario y Rojo-Negro tienen página de interacción.
4. Las operaciones disponibles se obtienen desde adaptadores.
5. El estado visual se actualiza después de cada operación.
6. El código de dominio no importa Flask.
7. Las plantillas reutilizan header, footer y navbar.
8. Existe ayuda didáctica del módulo y de cada estructura.
9. Existen pruebas iniciales.
10. El proyecto queda preparado para futura integración de TAD implementados en C.

Prohibiciones explícitas:

- No generar código monolítico.
- No poner lógica de estructuras en las rutas Flask.
- No poner lógica de negocio en JavaScript si debe estar en backend.
- No usar eval.
- No ignorar excepciones.
- No inventar métodos inexistentes en los TAD.
- No eliminar archivos TAD originales.
- No acoplar la UI directamente a una clase concreta del dominio.
- No implementar grafos.
- No implementar Union-Find.
- No afirmar rotaciones específicas si el TAD no expone dicha información.

Formato de salida esperado:

1. Primero crea un plan de implementación detallado antes de desarrollar la aplicación.
2. El plan debe incluir:
 - análisis de los TAD suministrados;
 - mapa entre operaciones de UI y métodos reales;
 - estructura final de carpetas;
 - fases de implementación;
 - componentes backend, frontend, servicios, adaptadores y pruebas;
 - riesgos técnicos;
 - operaciones pendientes o no soportadas;
 - criterios de validación antes de codificar.
3. Después de presentar el plan, desarrolla la aplicación siguiendo ese plan.
4. Muestra el árbol de carpetas creado.
5. Explica brevemente la responsabilidad de cada carpeta.
6. Implementa los archivos necesarios.
7. Incluye comentarios y docstrings.
8. Incluye pruebas.
9. Incluye instrucciones para ejecutar:
 - creación de entorno virtual;
 - instalación de dependencias;

- ejecución de Flask;
- ejecución de pruebas.

10. Indica claramente cualquier operación pendiente porque no exista en los TAD suministrados.

Antes de codificar:

Analiza los TAD suministrados y extrae sus clases, métodos y excepciones reales. Usa esa información para construir los adaptadores. Si existe discrepancia entre esta especificación y los TAD reales, respeta los TAD reales y documenta la diferencia.

19. Control de versiones

Rama principal

main

Debe contener solo versiones estables.

Rama de desarrollo

develop

Para integración de funcionalidades.

Ramas sugeridas

feature/hierarchical-module
feature/hierarchical-routes
feature/tree-adapters
feature/tree-visualizer
feature/heap-visualizer
feature/hierarchical-help
feature/hierarchical-tests

Convención de commits

feat: agrega módulo de estructuras jerárquicas
feat: agrega adaptador para ABB
feat: agrega visualizador de árboles
fix: corrige manejo de árbol vacío
docs: agrega ayuda didáctica para AVL
test: agrega pruebas para árbol rojo-negro
refactor: separa visualización de árboles y montículos

20. Decisiones arquitectónicas importantes

ID	Decisión	Justificación
DA-J-001	Separar dominio y presentación	Evita acoplamiento entre TAD y Flask
DA-J-002	Usar adaptadores	Permite normalizar operaciones y preparar integración con C
DA-J-003	Serializar estado visual	Facilita comunicación backend/frontend
DA-J-004	Usar plantillas base	Mantiene consistencia con el módulo secuencial
DA-J-005	Representar montículo de dos formas	Ayuda a entender su implementación real por arreglo
DA-J-006	No inventar rotaciones	Evita explicar eventos que el TAD no expone explícitamente
DA-J-007	Mantener TAD originales	Reduce riesgo de alterar implementaciones existentes
DA-J-008	Marcar pendientes técnicos	Evita que agentes IA inventen operaciones o datos no disponibles
DA-J-009	Preparar integración con C	Permite cambiar implementación sin rediseñar la UI